Kurs: HI1029 Algoritmer och datastrukturer
Ibrahim Awad

Inledning

I denna uppgift analyserar jag en kod med fyra nästlade loopar. Målet är att ta reda på hur många gånger satsen r++ körs beroende på värdet n. Denna funktion kallar jag $T(n)$. Analysen görs med numerisk metod genom att studera resultat för olika värden på n.

Metod

Jag bestämmer tidsfunktionen $T(n)$ med hjälp av numerisk analys i tre steg. Jag börjar med att köra programmet för flera olika värden på n. För varje värde räknar jag hur många gånger satsen r++ utförs.

Exempel på erhålla värden visas i tabellen nedan:

n	T(n)
10	1705 gånger
20	23 485 gånger
40	347 270 gånger
1000	125 miljarder gånger

I det första försöket anpassar jag polynom av olika grad till de numeriska värdena på $T(n)$. Jag börjar med polynom av låg grad och ökar graden steg för steg. För varje polynom jämför jag hur bra det passar de värden jag har räknat fram.

I det andra försöket använder jag den lägsta grad som fungera bra och anpassar ett polynom av just denna grad till alla datapunkter. Detta polynom betraktar jag som en tidsfunktionen $T(n)$.

I det tredje försöket skriver jag om polynomets koefficienter till bråkform. Slutligen verifierar jag resultatet genom att beräkna $T(1000)$ både genom direkt körning av programmet och genom att använda polynomet, och jämför dessa värden.

Resultat

Programmets körning för olika värden på n visar en växande följd av T(n)-värden. Tabellen visar tydligt att antalet exekveringar ökar snabbt när n blir större.

Ett polynom av grad 3 följer trenden men passar inte värdena tillräckligt bra, särskilt för större n. Detta visar att grad 3 inte räcker.

Ett polynom av grad 4 följer däremot värdena mycket bättre. Det finns inga tydliga skillnader mellan polynomet och de beräknade värdena. Därför är grad 4 den lägsta grad som behövs.

När jag skriver om polynomets koefficienter till bråkform bli resultatet:

$$\mathbf{T(n) = (1/8)n^4 + (5/12)n^3 + (3/8)n^2 + (1/12)n}$$

Eller med gemensamma nämnare:

$$\mathbf{T(n) = (3n^4 + 10n^3 + 9n^2 + 2n) / 24}$$

För att kontrollera extrapolering till n = 1000 då beräknar jag $T(1000)$ genom att köra programmet direkt. Då får jag

$$\boldsymbol{T(1000) = 125\ 417\ 041\ 750}$$ **ca 125 miljarder operationer**

Slutsats

Den numeriska analysen visar att tidsfunktionen för den koden växer mycket snabbt när n blir större. Genom att anpassa polynom av olika grad (grad 2, 3, 4 och 5) till de uppmätta värdena konstaterar jag att ett polynom av grad 4 är det lägsta som kan beskriva tillväxten med exakt överensstämmelse.

Den dominerande termen är n^4, vilket innebär att antalet exekveringar ökar proportionellt mot $\mathbf{n^4}$ för stora värden på **n**. Därför är kodens tidskomplexitet av ordningen $\mathbf{O(n^4)}$. Detta innebär att komplexiteter är $\mathbf{O(n^4)}$**: 125,000,000,000** operationer, vilket är ineffektivt vid större n.